COLLEGE DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
Annual Project Showcase & Hackathon Assessment

OFFICIAL PROJECT SUBMISSION REPORT

STUDY PLANNER

A MERN-Based Automated Academic Schedule Orchestrator & Completion Tracker

Problem Statement No.: 85

Subject Track:	Backend Web Development
Technology Stack:	MERN (MongoDB, Express.js, React.js, Node.js)
Target System:	RESTful API Architecture & Single Page App
Academic Evaluation:	Hackathon Technical Assessment & Viva Voce

SUBMITTED BY:

Team Member Name	Roll Number	Core Assignment
A. Rohith	25MVCSMR0456	Backend API, Express Controller & MongoDB Schema
K. Kamal	25MVCSMR0502	React Frontend, Client State & UI Integration

Verified and submitted for academic project review pursuant to official Hackathon instructions.
Academic Year: 2025–2026 | Department of Computer Applications & Software Engineering

TABLE OF CONTENTS

Chapter	Title and Topic Description	Page No.
—	Cover Page & Team Details	1
—	Table of Contents	2
1	Abstract	3
2	Introduction & Background	3
3	Assigned Problem Statement & Analysis	4
4	Project Objectives & Success Criteria	4
5	Scope of the Project	5
6	Existing System & Critical Drawbacks	5
7	Proposed System Architecture & Methodology	6
8	Functional and Non-Functional Requirements	6
9	System Architecture & Data Flow Diagram (DFD)	7
10	Technologies Used (MERN Stack Rationale)	8
11	Database Design & MongoDB Schema Specification	9
12	CRUD Operations Implementation & Workflow	10
13	REST API Design & Endpoint Specification	11
14	Algorithms, Validation & Implementation Methodology	11
15	User Interface Design & Implementation Placeholders	12
16	Testing, Test Cases, and Verification Matrix	13
17	Limitations & Pending Constraints	13
18	Future Enhancements (RBAC & Intelligent Rescheduling)	14
19	Conclusion	14
20	Official References & Technical Bibliography	14

Document Organization & Evaluation Guide

This report is structured in direct accordance with standard academic engineering and hackathon submission guidelines. Each chapter explains practical software engineering concepts using plain language suitable for viva voce examinations while providing comprehensive technical specifications, schema definitions, and verification matrices.

1. ABSTRACT

Academic success in rigorous computer science curricula relies on effective time management and disciplined coverage of syllabi. Students commonly struggle to organize study schedules across multiple concurrent subjects, leading to procrastination, missed deadlines, and poor exam preparedness. **Study Planner** (Problem Statement No. 85) is an end-to-end web application developed using the **MERN Stack** (MongoDB, Express.js, React.js, and Node.js) that allows learners to create structured study schedules and monitor topic completion in real time.

The platform enables students to record, categorize, update, and manage academic topics by capturing key attributes: *subject, topic, date, allocated time, and completion status*. The backend provides a decoupled RESTful API built on Node.js and Express.js, persisting data within a flexible MongoDB database. The frontend utilizes React.js to present an interactive Single Page Application (SPA) featuring immediate visual feedback and dynamic completion progress metrics. This report details the complete software engineering lifecycle of the project, including architectural design, database modeling, API contracts, verification testing, and future enhancement roadmaps.

2. INTRODUCTION

Undergraduate engineering students handle diverse subjects spanning systems programming, algorithms, database management, and web engineering. Managing independent study hours across these disparate disciplines through paper planners or unstructured reminder apps frequently results in poor tracking and neglected modules.

The **Study Planner** project addresses this organizational challenge by implementing a unified digital planner. Built as part of the Backend Web Development course track, the core architectural goal is to demonstrate robust Full-Stack JavaScript principles. By unifying the programming language across client and server tiers through JavaScript (ES6+), the system achieves frictionless JSON data interchange, streamlined asynchronous request cycles, and modular maintainability.

Core Hackathon Project Identity

Title: Study Planner | **Track:** Backend Web Development | **Assigned ID:** PS-85.
Primary Mandate: Manage subject, topic, date, time, and status using MongoDB; develop Express.js/Node.js REST APIs; construct a reactive React UI; and clearly demonstrate CRUD mechanics and completion monitoring.

3. PROBLEM STATEMENT & ANALYSIS

> **Official Hackathon Problem Statement (No. 85):**
>
> *"Build a MERN-based study planner that creates study schedules and monitors completion. The system should manage subject, topic, date, time, and status using MongoDB. React should provide the user interface, while Node.js and Express.js should handle the backend APIs. The system should support adding, viewing, updating, and managing relevant records."*

A rigorous breakdown of the problem statement reveals five critical technical requirements:

- **Data Granularity:** The database must rigorously persist five mandatory data attributes for every study item: `subject`, `topic`, `date`, `time`, and `status`.
- **Architectural Separation:** Strict separation of concerns between presentation (React) and persistence/business logic (Node.js/Express.js).
- **Schedule Generation:** Capabilities to allocate concrete calendar dates and time durations to academic topics.
- **Completion Monitoring:** Mechanisms to track transition states (e.g., Pending, In-Progress, Completed) and calculate real-time completion percentages.
- **Record Management:** Comprehensive CRUD (Create, Read, Update, Delete) capability exposed via documented HTTP endpoints.

4. PROJECT OBJECTIVES

To fulfill both course learning outcomes and hackathon benchmarking criteria, the development team executed the following six explicit objectives:

1. **Demonstrate MERN Stack Integration:** Establish a unified development pipeline coupling MongoDB NoSQL storage, Express.js middleware routing, Node.js runtime environments, and React component state management.
2. **Implement Robust MongoDB CRUD Operations:** Author formal Mongoose schemas with data type constraints, required fields, and built-in validations.
3. **Construct RESTful Backend APIs:** Implement clean HTTP endpoints adhering to standard conventions (`GET`, `POST`, `PUT/PATCH`, `DELETE`) returning structured JSON responses.
4. **Engineer a Responsive React Frontend:** Construct intuitive forms, dynamic scheduling lists, and live status toggle triggers with client-side state synchronization.
5. **Address Security, Authentication & Access Control:** Assess authentication and role-based access requirements; establish architectural hooks and outline migration paths for token-based security.
6. **Deliver Transparent Results & Metrics:** Compute and render live completion analytics (total tasks, finished tasks, pending tasks, percentage complete) to foster student accountability.

5. SCOPE OF THE PROJECT

The scope of this project defines the operational boundaries implemented during the hackathon development cycle:

- **In-Scope Capabilities:** Creation of discrete study items; classification by academic subject; date and time duration allocation; multi-state tracking (`Pending`, `In Progress`, `Completed`); topic modification and deletion; and real-time computation of topic completion metrics across the active schedule.
- **Target Audience:** Undergraduate college students, competitive examination aspirants, and self-directed technical learners needing structured daily or weekly study plans.
- **Deployment Context:** Localhost and staging cloud environments (Node.js runtime, MongoDB Atlas cloud cluster) accessible through standard modern web browsers.
- **Out-of-Scope (Phase 1):** Multi-tenant billing, automated natural-language syllabus parsing, external calendar (Google/Outlook) two-way synchronization, and native mobile compilation (targeted for future versions).

6. EXISTING SYSTEM & CRITICAL DRAWBACKS

Traditionally, college students utilize physical notebooks, paper diaries, static spreadsheet tables (e.g., Microsoft Excel, Google Sheets), or generic to-do mobile applications to track academic preparation. An investigation of these manual and semi-digital approaches highlights multiple failure points:

Tracking Method	Observed Operational Drawbacks	Impact on Academic Performance
Physical Paper Planners	Prone to loss; no automated completion metrics; static layout prevents easy rescheduling of missed topics.	High rate of abandoned schedules; zero analytical visibility into revision backlogs.
Spreadsheets (Excel)	Clunky mobile interfaces; manual formula maintenance required; no native alerts or streamlined status toggling.	Excessive administrative friction; students spend time formatting sheets rather than studying.
Generic To-Do Apps	Lacks academic context (subject/topic hierarchy); does not capture dedicated study durations; lacks syllabus progress metrics.	Fails to provide subject-wise syllabus coverage insights before exams.

These deficiencies establish an urgent requirement for a dedicated, web-based platform tailored specifically to academic workflows.

7. PROPOSED SYSTEM ARCHITECTURE & METHODOLOGY

The proposed **Study Planner** overcomes the deficiencies of traditional methods by providing an integrated, real-time client-server web solution. Built on the MERN stack, the system centralizes schedule creation, dynamic date/time allocation, and completion tracking into a single reactive interface.

Whenever a student adds or updates a study task, React transmits an asynchronous HTTP request via the Axios library to an Express.js router hosted on Node.js. The backend validates the payload and performs ACID-compliant atomic operations on the MongoDB document store. The updated data is immediately mirrored back to the React UI without requiring full page reloads, ensuring instant feedback and high responsiveness.

8. FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

8.1 Functional Requirements (FR)

Req ID	Feature Module	Functional Specification
FR-01	Schedule Creation	System must accept Subject, Topic, Scheduled Date, Time Duration, and initial Status via a validated form.
FR-02	Schedule Viewing	System must query and display all persisted study records in sorted chronological order.
FR-03	Record Updation	System must allow modification of scheduled dates, durations, and direct status toggling (Pending → Completed).
FR-04	Record Deletion	System must permit removal of redundant or obsolete study records with immediate database deletion.
FR-05	Completion Monitoring	System must calculate total tasks, completed tasks, and compute completion percentage dynamically.

8.2 Non-Functional Requirements (NFR)

- **Performance & Latency:** API response times must remain under 200 milliseconds for standard CRUD operations under single-user simulation.
- **Reliability & Persistence:** Zero data loss on server restarts through persistent MongoDB cloud storage and connection auto-retry policies.
- **Usability & Simplicity:** The user interface must be intuitive, requiring zero training for a student to plan their first subject session.
- **Maintainability:** Decoupled architectural structure following standard MVC (Model-View-Controller / Model-Route-Controller) patterns.

9. SYSTEM ARCHITECTURE & DATA FLOW DIAGRAM

9.1 High-Level MERN System Architecture



The system employs a classic 3-tier architecture. The Presentation Layer (React) runs entirely in the user's browser, communicating asynchronously via JSON over HTTP. The Application Layer (Node.js/Express) parses incoming routes and validates inputs. The Data Layer (MongoDB) guarantees durable document storage.

9.2 Data Flow Diagram (DFD Level-1)



As mapped above, user inputs pass through validation processes before persistent write operations occur in data store D1. Subsequent status alterations trigger calculation routines that return aggregated completion ratios back to the client interface.

10. TECHNOLOGIES USED (MERN STACK RATIONALE)

The project strictly leverages the **MERN Stack** as designated by Problem Statement No. 85. Below is the functional explanation of each technology's role within this academic project:

10.1 MongoDB (Database Layer)

MongoDB is a source-available, cross-platform, document-oriented NoSQL database. Rather than storing records in rigid relational tables with strict foreign-key dependencies, MongoDB stores study records as JSON-like documents with dynamic schemas (BSON format). In the Study Planner project, MongoDB stores topic definitions, dates, and completion flags. It was selected because academic tasks vary in structure (some require descriptive revision notes, others only basic time allocations), and its native JSON handling completely eliminates object-relational impedance mismatch.

10.2 Express.js (Backend Web Framework)

Express.js is a minimalist web application framework running on Node.js. It acts as the routing and middleware pipeline for the Study Planner backend. Express handles incoming HTTP requests (such as `POST /api/schedules`), passes payloads through body-parsing middleware (`express.json()`), verifies request validity, executes business logic, and delegates database queries to Mongoose models.

10.3 React.js (Frontend User Interface)

React.js is an open-source declarative JavaScript library maintained by Meta for building dynamic user interfaces. The Study Planner frontend is constructed as a Single Page Application (SPA). React manages internal component state (using hooks like `useState` and `useEffect`). When a task status changes from "Pending" to "Completed", React re-renders only the affected DOM nodes rather than reloading the entire webpage, delivering fluid user feedback.

10.4 Node.js (JavaScript Runtime Environment)

Node.js is an asynchronous, event-driven JavaScript runtime built on Chrome's V8 engine. Node.js executes the backend server code outside of the browser. Its non-blocking, event-driven I/O model makes it lightweight and capable of servicing concurrent API requests from multiple student sessions efficiently without thread-locking bottlenecks.

Technology Component	Role in Study Planner	Primary Module / Dependency Used
MongoDB	Primary persistent data storage for study records	MongoDB Community / Atlas, Mongoose ODM
Express.js	HTTP routing, request parsing & API dispatching	`express`, `cors`, `dotenv`
React.js	Client interface, forms, and completion graphs	`react`, `react-dom`, `axios`
Node.js	Server runtime environment hosting the API	Node LTS (v18.x / v20.x), NPM

11. DATABASE DESIGN & SCHEMA SPECIFICATION

The data model for the Study Planner is designed around the exact requirements of Problem Statement No. 85, which dictates managing **subject, topic, date, time, and status** using MongoDB.

Data modeling is implemented using the **Mongoose Object Data Modeling (ODM)** library. Mongoose enforces strict schema definitions and validation rules at the application layer while preserving MongoDB's underlying document flexibility.

11.1 Schedule Entity Schema Field Dictionary

Field Name	BSON Type	Validation Rule / Constraints	Description & Purpose
`_id`	ObjectId	Auto-generated primary key (UUID)	Unique identifier for every study record.
`subject`	String	`required: true, trim: true`	Academic course title (e.g., "Operating Systems").
`topic`	String	`required: true, trim: true`	Specific chapter or unit (e.g., "Deadlock Avoidance").
`date`	Date	`required: true` (ISO 8601)	Target calendar study date (YYYY-MM-DD).
`time`	String / Number	`required: true` (e.g., "2 Hours" / 120 min)	Allocated study duration or planned daily time slot.
`status`	String	Enum: `['Pending', 'In Progress', 'Completed']`	Tracks academic completion; default is `'Pending'`.
`createdAt`	Date	Auto-populated timestamp	Audit timestamp indicating task generation time.

11.2 Mongoose Schema Implementation Snippet

```
// models/Schedule.js - Official Study Planner Schema Definition
const mongoose = require('mongoose');

const scheduleSchema = new mongoose.Schema({
  subject: { type: String, required: [true, 'Subject is required'], trim: true },
  topic:   { type: String, required: [true, 'Topic is required'], trim: true },
  date:    { type: Date, required: [true, 'Study date is required'] },
  time:    { type: String, required: [true, 'Allocated study time is required'] },
  status:  {
    type: String,
    enum: ['Pending', 'In Progress', 'Completed'],
    default: 'Pending'
  }
}, { timestamps: true });

module.exports = mongoose.model('Schedule', scheduleSchema);
```

12. CRUD OPERATIONS IMPLEMENTATION & WORKFLOW

The core of Problem Statement No. 85 requires the system to support **adding, viewing, updating, and managing** study records. In backend web development, this translates directly to the four foundational CRUD (Create, Read, Update, Delete) operations.

12.1 CREATE (Adding Study Tasks)

Client Trigger: The student completes the React input form with the subject name, chapter topic, calendar date, and duration, then clicks "Add Schedule".

API & Database Action: Axios dispatches a `POST` request containing the JSON payload to `/api/schedules`. Express runs `Schedule.create(req.body)`. MongoDB verifies non-null fields, generates an `_id`, and commits the document, returning HTTP status `201 Created`.

12.2 READ (Viewing & Fetching Schedules)

Client Trigger: The React app mounts, triggering a `useEffect` hook that dispatches a `GET` request.

API & Database Action: The Express route invokes `Schedule.find().sort({ date: 1 })`. MongoDB queries the collection and returns an array of sorted documents. React updates its state and maps over the array to render the study table, returning HTTP status `200 OK`.

12.3 UPDATE (Status Toggling & Schedule Edits)

Client Trigger: The student clicks the status badge to switch from "Pending" to "Completed", or edits topic details via a modal form.

API & Database Action: React dispatches a `PUT` or `PATCH` request to `/api/schedules/:id`. Express executes `Schedule.findByIdAndUpdate(req.params.id, req.body, { new: true, runValidators: true })`. MongoDB updates the document in place and returns the updated record, triggering live UI metric re-calculations.

12.4 DELETE (Record Removal & Cleanup)

Client Trigger: The student clicks the "Delete" icon next to an obsolete or canceled study topic.

API & Database Action: An HTTP `DELETE` request is transmitted to `/api/schedules/:id`. The server executes `Schedule.findByIdAndDelete(req.params.id)`. MongoDB permanently deletes the record, and the frontend filters out the deleted item from local state without full reload.

13. REST API DESIGN & ENDPOINT SPECIFICATION

The backend delivers a stateless RESTful Web API built following standard HTTP semantics. The endpoints are organized under the base path `/api/schedules`.

Method	Endpoint	Description & Request Body	Expected Status
GET	`/api/schedules`	Retrieve all study records sorted by target date.	`200 OK`
GET	`/api/schedules/:id`	Fetch a single schedule record by its unique ObjectId.	`200 OK / 404`
POST	`/api/schedules`	Create record. Body: `{subject, topic, date, time, status}`	`201 Created`
PUT	`/api/schedules/:id`	Update all or specific fields of an existing study item.	`200 OK / 400`
DELETE	`/api/schedules/:id`	Permanently remove a study record from the collection.	`200 OK / 404`

14. ALGORITHMS & IMPLEMENTATION METHODOLOGY

14.1 Completion Percentage Computation Algorithm

The core requirement to "monitor completion" is computed client-side in React upon receiving the schedule array from the backend. The mathematical algorithm iterates through the record collection to derive dynamic completion metrics:

```
// Algorithm: Dynamic Schedule Completion Calculation
totalTasks = schedules.length;
completedTasks = schedules.filter(item => item.status === 'Completed').length;
pendingTasks = totalTasks - completedTasks;
completionRate = totalTasks > 0 ? ((completedTasks / totalTasks) * 100).toFixed(1) : 0;
```

This algorithm executes in linear time *O(N)* relative to task count, providing instantaneous metric calculations with zero rendering lag.

14.2 Input Validation & Error Sanitization

Both client and server enforce defense-in-depth data validation. The React form blocks submission if mandatory text fields are blank or dates are malformed. The Express controller verifies incoming payloads using Mongoose validators, ensuring invalid records are rejected with descriptive JSON error responses (e.g., `{ success: false, error: "Subject is required" }`).

15. USER INTERFACE DESIGN & IMPLEMENTATION

The frontend is built using React components organized into modular, reusable sections: the Navigation Bar, the Task Creation Form, the Metric Progress Banner, and the Interactive Schedule Data Table.

15.1 UI Layout & Screenshot Placeholders

Per official evaluation guidelines, placeholders are provided below for pasting high-resolution screenshots of the running React web interface during final evaluation:

[SCREENSHOT PLACEHOLDER 1: MAIN DASHBOARD & SCHEDULE ENTRY FORM]

Paste capture showing: Subject input, Topic field, Date picker, Time allocation dropdown, and "Add to Schedule" action button alongside the real-time completion progress bar.

[SCREENSHOT PLACEHOLDER 2: STUDY SCHEDULE TABLE & STATUS TOGGLING]

Paste capture showing: Rendered MongoDB documents in table view with columns (Subject, Topic, Date, Duration, Status, Actions) with clickable 'Pending' / 'Completed' badges and 'Delete' triggers.

15.2 Client-Side State Management

React manages application flow through localized state hooks without requiring external heavyweight libraries like Redux. The central state array `schedules` stores all retrieved tasks. When a user marks an item completed, an optimistic UI update immediately alters the local state badge color to green (Completed) while dispatching the background `PUT` request to persist the change in MongoDB.

16. TESTING, TEST CASES & VERIFICATION MATRIX

Comprehensive manual and API verification tests were conducted across all CRUD endpoints and user interface interactions to validate stability and compliance with Problem Statement No. 85.

Test ID	Test Scenario	Input / Action	Expected Result	Status
TC-01	Valid Schedule Creation	Submit form with: "OS", "Semaphores", "2026-03-20", "2 hrs".	Record saved; 201 Created returned; table updates.	Passed
TC-02	Missing Required Fields	Submit form with blank Subject or Topic.	Submission rejected; Mongoose validation error raised.	Passed
TC-03	Fetch All Records	Send GET request to `/api/schedules`.	Returns JSON array with all records sorted by date.	Passed
TC-04	Status Toggle Update	Click status toggle: Pending → Completed.	PUT request succeeds; progress bar increments.	Passed
TC-05	Delete Study Record	Click delete button on specific task row.	DELETE request returns 200; item removed from view.	Passed

17. SYSTEM LIMITATIONS

In strict adherence to academic honesty and hackathon verification instructions, the team clearly documents the practical boundaries and current limitations of the system:

- **Single-User Architecture:** The current database model does not isolate records per authenticated user. All schedules currently reside in a shared collection accessible to the active client.
- **Absence of Native Calendar Alerts:** The application does not currently trigger push notifications, SMS alerts, or automated email reminders when a scheduled study time arrives.
- **Manual Rescheduling:** Overdue topics marked "Pending" do not automatically roll over to the following day without manual user intervention.

18. FUTURE ENHANCEMENTS & SECURITY ROADMAP

Building upon the verified core CRUD foundation, the system architecture has been prepared for several critical future upgrades:

- **Authentication & Role-Based Access Control (RBAC):** (*Pending Requirement*) Integrating JSON Web Tokens (JWT) and BCrypt password hashing to provide isolated individual student accounts and specialized "Mentor/Tutor" roles capable of reviewing student completion progress.
- **Intelligent Schedule Rollover:** Implementing automated cron jobs via Node-Cron to flag past-due pending tasks and automatically propose rescheduled revision dates.
- **External Calendar Integration:** Providing bi-directional iCalendar (.ics) and Google Calendar synchronization for automated smartphone schedule alerts.
- **Analytical Visualizations:** Embedding graphical charts (e.g., Chart.js) depicting subject-wise study hour distributions and weekly consistency heatmaps.

19. CONCLUSION

The **Study Planner** web application successfully solves the assigned Hackathon Problem Statement No. 85 by engineering a clean, full-stack MERN solution for academic planning. The application delivers robust data management across subjects, topics, dates, allocated times, and completion statuses. By establishing a decoupled REST API on Express.js and Node.js with MongoDB persistence, the backend guarantees reliable data integrity, while the reactive React frontend provides immediate visual tracking of academic progress. The project demonstrates practical competency in full-stack JavaScript architecture and establishes a production-grade foundation for future educational technology initiatives.

20. REFERENCES

1. MongoDB Inc., *MongoDB Documentation: The Developer Data Platform Manual*, 2024. [Online]. Available: https://www.mongodb.com/docs/
2. Express.js Foundation, *Express - Fast, unopinionated, minimalist web framework for Node.js*, 2024. [Online]. Available: https://expressjs.com/
3. Meta Platforms, Inc., *React Documentation: Describing the UI & Managing State*, 2024. [Online]. Available: https://react.dev/
4. OpenJS Foundation, *Node.js v20 LTS Runtime Environment Documentation*, 2024. [Online]. Available: https://nodejs.org/docs/
5. Automattic, *Mongoose ODM v8: Elegant MongoDB Object Modeling for Node.js*, 2024. [Online]. Available: https://mongoosejs.com/docs/